Rapport — Linux From Scratch (LFS) 13.0-systemd

> **Système cible :** Debian (machine hôte) **Partition LFS :** `/mnt/lfs` (15 Go, ext4 sur `/dev/sda3`)
> **Versions notables :** Binutils 2.46.0 · GCC 15.2.0 · Linux 6.18.10 · Glibc 2.43 · Systemd 259.1 **Date de rédaction :** 2026-04-06

Sommaire

1. Préparation de la machine hôte
2. Vérification des dépendances
3. Partitionnement et montage
4. Structure de répertoires LFS
5. Utilisateur dédié `lfs`
6. Environnement de construction
7. Compilation croisée — Chapitre 5
    - 7.1 Binutils pass 1
    - 7.2 GCC pass 1
    - 7.3 Linux API Headers
    - 7.4 Glibc
    - 7.5 Libstdc++
8. Outils temporaires croisés — Chapitre 6
    - 8.1 M4
    - 8.2 Ncurses
    - 8.3 Bash
    - 8.4 Coreutils
    - 8.5 Diffutils
    - 8.6 File
    - 8.7 Findutils
    - 8.8 Gawk
    - 8.9 Grep
    - 8.10 Gzip
    - 8.11 Make
    - 8.12 Patch
    - 8.13 Sed
    - 8.14 Tar
    - 8.15 Xz
    - 8.16 Binutils pass 2
    - 8.17 GCC pass 2
9. Entrée dans le Chroot — Chapitre 7
10. Construction du système final — Chapitre 8
    - 10.1 Vue d'ensemble
    - 10.2 Tableau des 83 paquets
    - 10.3 Paquets notables
    - 10.4 Nettoyage et stripping

11. Configuration du système — Chapitre 9
    - 11.1 Réseau (systemd-networkd)
    - 11.2 Hostname et /etc/hosts
    - 11.3 Horloge et fuseau horaire
    - 11.4 Console virtuelle
    - 11.5 Locale
    - 11.6 /etc/inputrc
    - 11.7 /etc/shells
12. Kernel et Bootloader — Chapitre 10
    - 12.1 /etc/fstab
    - 12.2 Compilation du kernel
    - 12.3 Installation de GRUB
    - 12.4 Fichiers d'identification
13. Démontage et redémarrage — Chapitre 11
14. État des scripts et ordre d'exécution

---

1. Préparation de la machine hôte

LFS se construit depuis un système Linux existant appelé **système hôte**. Ce système hôte fournit le compilateur, les outils de build et les bibliothèques nécessaires pour amorcer la chaîne de compilation — jusqu'à ce que le système LFS soit suffisamment autonome pour se compiler lui-même. Debian a été choisi comme hôte car c'est une distribution stable, bien documentée, dont les paquets de développement sont facilement disponibles.

```
su root
apt install sudo
sudo apt update
sudo usermod -aG sudo faucheur
```

> **Note :** Se déconnecter et se reconnecter après `usermod` pour que le groupe `sudo` soit pris en compte dans la session.

---

2. Vérification des dépendances

Avant de commencer, LFS exige que l'hôte dispose d'un ensemble minimal d'outils de build. Ces outils ne seront utilisés **que** pour construire la première passe de la chaîne croisée — le système LFS final les recompilera lui-même depuis les sources. Si une version est trop ancienne ou un outil manquant, certaines compilations échoueront de manière obscure (erreurs de syntaxe, symboles manquants, scripts configure qui s'arrêtent sans raison apparente).

```
sudo apt update
sudo apt install -y \
  build-essential \
  binutils \
  bison \
```

```
    gawk \
    m4 \
    patch \
    texinfo \
    perl \
    python3 \
    flex \
    libgmp-dev \
    libmpfr-dev \
    libmpc-dev \
    wget \
    xz-utils \
    bzip2
```

Lien symbolique requis : `/bin/sh` doit pointer vers `/bin/bash`, car les scripts de build LFS utilisent des fonctionnalités spécifiques à Bash que le shell POSIX minimal (`dash`, défaut sur Debian) ne supporte pas.

```
ln -sfv bash /bin/sh
```

Vérifications à valider :

Élément	Cible
Shell actif	`bash`
`/bin/sh`	lien symbolique → `bash`
`/usr/bin/awk`	lien symbolique → `gawk`
`/usr/bin/yacc`	lien symbolique → `bison` (ou script wrapper)
`gcc --version`	GCC ≥ 5.1
`glibc --version`	Glibc ≥ 2.11
`python3 --version`	Python ≥ 3.4

> **Attention :** Si `/usr/bin/awk` pointe vers `mawk` (défaut sur certains Debian), certains tests de compilation échoueront silencieusement. Vérifier avec `ls -la /usr/bin/awk`.

---

3. Partitionnement et montage

LFS est construit sur une partition dédiée, complètement séparée du système hôte. Cette isolation est importante : elle garantit que les fichiers produits ne se mélangent pas avec l'hôte, et que le futur système peut être démarré indépendamment. La taille de 15 Go est suffisante pour le système de base LFS avec une marge confortable pour les sources temporaires (~5 Go pendant le build).

État initial du disque :

```
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   35G  0 disk
├─sda1   8:1    0 18,9G  0 part /
├─sda2   8:2    0    1K  0 part
└─sda5   8:5    0  1,1G  0 part [SWAP]
```

Création de la partition `sda3` (15 Go) via `cfdisk`, puis formatage en ext4 :

```
sudo cfdisk /dev/sda
sudo mkfs -v -t ext4 /dev/sda3
```

La variable `LFS` est la pierre angulaire de toute l'installation. Elle indique à chaque script et chaque commande où se trouve la racine du futur système. **Toute commande utilisant `$LFS` sans que cette variable soit définie opère directement sur l'hôte.**

```
export LFS=/mnt/lfs
mkdir -pv $LFS
mount -v -t ext4 /dev/sda3 $LFS
```

État final :

```
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   35G  0 disk
├─sda1   8:1    0 18,9G  0 part /
├─sda2   8:2    0    1K  0 part
├─sda3   8:3    0   15G  0 part /mnt/lfs
└─sda5   8:5    0  1,1G  0 part [SWAP]
```

> **Attention :** La variable `LFS` et le montage de `/dev/sda3` sont perdus à chaque redémarrage. Il faut **toujours** les reconfigurer avant de reprendre le travail :
>
> ```
> export LFS=/mnt/lfs
> mount -v -t ext4 /dev/sda3 $LFS
> ```

---

4. Structure de répertoires LFS

Avant de commencer à compiler quoi que ce soit, il faut créer le squelette de répertoires que le futur système LFS utilisera. Ce squelette suit la norme **FHS** (Filesystem Hierarchy Standard) qui définit où chaque type de fichier doit se trouver sur un système Linux (`/usr/bin` pour les exécutables, `/etc` pour la configuration, etc.).

Un point subtil : sur un système Linux 64 bits moderne, `/bin`, `/lib` et `/sbin` sont des liens symboliques vers `/usr/bin`, `/usr/lib` et `/usr/sbin`. LFS adopte cette convention dès le départ pour éviter d'avoir deux emplacements différents pour les mêmes fichiers.

```
sudo mkdir -pv $LFS/{etc,var} $LFS/usr/{bin,lib,sbin}
mkdir -v $LFS/sources
chmod -v a+wt $LFS/sources   # sticky bit comme /tmp

for i in bin lib sbin; do
  sudo ln -sv usr/$i $LFS/$i
done

case $(uname -m) in
  x86_64) sudo mkdir -pv $LFS/lib64 ;;
esac

sudo mkdir -pv $LFS/tools
```

Le répertoire `$LFS/tools` est temporaire : il accueillera la chaîne de compilation croisée du chapitre 5. Il sera supprimé une fois le système final construit. Le répertoire `$LFS/sources` contiendra toutes les archives des paquets sources.

> **Incident critique rencontré :** La commande `ln -sv usr/$i $LFS/$i` a été exécutée après un redémarrage sans que `$LFS` soit défini. La variable était vide, ce qui a provoqué l'écrasement de `/usr/bin`, `/usr/lib` et `/usr/sbin` sur le **système hôte**. La machine a dû être réinstallée depuis zéro.
>
> **Règle absolue :** Toujours vérifier `echo $LFS` avant toute commande impliquant cette variable.

---

5. Utilisateur dédié `lfs`

Toutes les compilations des chapitres 5 et 6 sont réalisées sous un utilisateur non-privilégié nommé `lfs`. L'idée est simple : un utilisateur sans droits root ne peut pas, même par erreur, écraser des fichiers système de l'hôte. Si une commande `make install` se retrouve avec un chemin incorrect, elle échouera avec une erreur de permission plutôt que de corrompre silencieusement le système hôte.

```
groupadd lfs
useradd -s /bin/bash -g lfs -m -k /dev/null lfs
sudo passwd lfs

chown -v lfs $LFS/{usr{,/*},var,etc,tools,sources}
case $(uname -m) in
  x86_64) chown -v lfs $LFS/lib64 ;;
esac
```

Le flag `-k /dev/null` évite que `useradd` copie les fichiers de `/etc/skel` dans le home de `lfs` — on veut un environnement vierge que l'on configure entièrement à la prochaine étape.

6. Environnement de construction

L'environnement shell de l'utilisateur `lfs` est délibérément minimaliste. L'objectif est d'éliminer toute variable d'environnement héritée de la session hôte qui pourrait interférer avec la compilation croisée. Des variables comme `CFLAGS`, `LD_LIBRARY_PATH`, ou `PKG_CONFIG_PATH` définies sur l'hôte pourraient rediriger le compilateur vers des bibliothèques hôtes au lieu des bibliothèques LFS — corrompant silencieusement le système en construction.

Pour comprendre pourquoi c'est critique : quand on compilera GCC ou Glibc dans les prochaines étapes, les scripts `configure` et les Makefiles explorent l'environnement pour décider où trouver les headers, les bibliothèques, les outils. Si `PKG_CONFIG_PATH` de l'hôte Debian pointe vers `/usr/lib/x86_64-linux-gnu/pkgconfig`, les paquets LFS risquent de lier contre des bibliothèques Debian à des chemins absolus qui n'existeront plus dans le système final — produisant des binaires cassés qui ne démarrent pas.

Quels fichiers bash lit-il et quand ?

Avant de détailler le contenu des fichiers, il est important de comprendre **quand bash les lit** — car beaucoup de problèmes viennent d'une confusion entre login shell et non-login shell.

Type de shell	Comment il est lancé	Fichiers lus
Login interactif	`su - lfs`, `ssh user@host`, login terminal	`/etc/profile`, puis `~/.bash_profile` (ou `~/.profile`)
Non-login interactif	Nouveau terminal dans une session existante, `bash` depuis un shell	`~/.bashrc`
Non-interactif	Scripts (`bash script.sh`), sous-shells	Aucun de ces fichiers (sauf si `$BASH_ENV` est défini)

Dans LFS, on exploite cette mécanique précisément :

- `.bash_profile` est lu une seule fois à la connexion (`su - lfs`) — il nettoie l'environnement et relance bash proprement.
- Le bash relancé est non-login, donc il lit `.bashrc` — qui définit toutes les variables de build.
- Tout sous-shell lancé pendant la compilation (par `make`, par les scripts `configure`) héritera des variables exportées, mais ne relira pas `.bash_profile`.

`.bash_profile` — shell épuré

`.bash_profile` est lu par bash uniquement lors des **connexions interactives login** (`su - lfs`, SSH, login terminal). C'est le point d'entrée principal de la session `lfs`. Son rôle est de démarrer un shell complètement propre avant que quoi que ce soit d'autre se produise.

```
cat > ~/.bash_profile << "EOF"
exec env -i HOME=$HOME TERM=$TERM PS1='\u:\w\$ ' /bin/bash
```

```
EOF
```

Décryptage mot à mot :

- **`exec`** : remplace le processus bash courant par un nouveau processus, au lieu d'en créer un fils. Concrètement, si on faisait `env -i bash` sans `exec`, on aurait deux processus bash empilés — le premier (avec l'environnement Debian pollué) attendrait que le second se termine, et quand on ferait `exit`, on retournerait dans le premier avec toutes ses variables. Avec `exec`, le premier processus est purement et simplement remplacé : il disparaît, il n'y a plus qu'un seul bash, propre.

- **`env -i`** : la commande `env` avec le flag `-i` (ignore environment) démarre le programme suivant avec un environnement **entièrement vide**. Aucune variable n'est transmise, même pas `PATH`. C'est le niveau d'isolation maximal.

- **`HOME=$HOME`** : la valeur de `HOME` est capturée *avant* que `env -i` vide l'environnement (car elle est évaluée dans le shell courant), et transmise au nouveau bash. Sans `HOME`, bash ne saurait pas où trouver `~/.bashrc`, et l'opérateur tilde `~` ne fonctionnerait pas.

- **`TERM=$TERM`** : de même, `TERM` décrit le type de terminal en cours (`xterm-256color`, `vt100`, etc.). Sans lui, les programmes basés sur ncurses (comme l'éventuel `make menuconfig` pour le kernel) ne pourraient pas déterminer les capacités du terminal et refuseraient de démarrer, ou afficheraient des artefacts visuels.

- **`PS1='\u:\w\$ '`** : définit le prompt du shell. `\u` affiche le nom d'utilisateur (`lfs`), `\w` affiche le répertoire courant. Exemple : `lfs:/mnt/lfs/sources$ ` . Sans ce paramètre, bash utiliserait un prompt par défaut ou vide.

Quand ce nouveau bash démarre, il lira automatiquement `~/.bashrc` (convention bash pour les shells interactifs non-login), qui définira les variables de compilation.

> **Note :** `exec` est crucial. Voici la différence concrète :
>
> - **Sans `exec`** : `env -i bash` → nouveau shell propre (enfant) → on travaille → `exit` → retour au shell parent pollué
> - **Avec `exec`** : le shell parent *devient* le shell propre → `exit` ferme directement la session

---

`.bashrc` — variables de compilation

`.bashrc` est sourcé par chaque shell bash interactif non-login. C'est ici que sont définies toutes les variables qui gouvernent le comportement de la compilation croisée.

```
cat > ~/.bashrc << "EOF"
set +h
umask 022
LFS=/mnt/lfs
LC_ALL=POSIX
LFS_TGT=$(uname -m)-lfs-linux-gnu
PATH=/usr/bin
if [ ! -L /bin ]; then PATH=/bin:$PATH; fi
```

```
PATH=$LFS/tools/bin:$PATH
CONFIG_SITE=$LFS/usr/share/config.site
export LFS LC_ALL LFS_TGT PATH CONFIG_SITE
EOF
```

Détail de chaque directive

`set +h` — désactivation du hash de commandes

Bash maintient en interne un cache (table de hash) qui mémorise le chemin complet de chaque commande la première fois qu'elle est appelée, pour éviter de parcourir `$PATH` à chaque appel. Ce comportement est normalement une optimisation utile, mais ici il est dangereux.

Voici pourquoi : au chapitre 5, on installe progressivement les outils LFS dans `$LFS/tools/bin`. Le problème est que bash peut avoir mémorisé le chemin d'un outil hôte avant qu'on installe la version LFS. Par exemple :

1. On invoque `ld` → bash cherche dans `$PATH`, trouve `/usr/bin/ld` (le linker Debian), le met en cache
2. On installe Binutils pass 1 → `$LFS/tools/bin/x86_64-lfs-linux-gnu-ld` apparaît dans `$PATH` en priorité
3. On invoque de nouveau `ld` → bash utilise le cache : `/usr/bin/ld` — **le mauvais**

`set +h` désactive ce cache. Bash parcourt `$PATH` à chaque appel de commande, ce qui est légèrement plus lent mais garantit qu'on obtient toujours la version la plus récente et la plus prioritaire.

> **Exemple de symptôme sans `set +h` :** Des erreurs comme `cannot find -lgcc_s` ou `undefined reference to __stack_chk_fail` lors de la compilation de Glibc, causées par le linker hôte qui ne connaît pas les bibliothèques LFS.

`umask 022` — masque de permissions

Le umask (user file creation mask) est un masque octal qui définit les permissions à *retirer* lors de la création de tout fichier ou répertoire. Il fonctionne en soustraction : les bits à 1 dans le umask sont retirés des permissions.

- Fichier : permissions max théoriques = 666 (rw-rw-rw-). Avec umask 022 : 666 - 022 = **644** (rw-r--r--)
- Répertoire : permissions max théoriques = 777 (rwxrwxrwx). Avec umask 022 : 777 - 022 = **755** (rwxr-xr-x)

Cela garantit que les fichiers installés par les paquets (bibliothèques `.so`, headers `.h`, binaires) sont lisibles et exécutables par tous les utilisateurs — y compris `root` qui compilera le chapitre 8, et l'utilisateur `tester` qui lancera les suites de tests.

> **Risque si umask est trop restrictif :** Un `umask 077` (parfois défaut sur des systèmes sécurisés) créerait des fichiers en 600 (rwx------), lisibles uniquement par leur propriétaire. Conséquence : lors de la compilation de GCC au chapitre 8, les headers de Glibc installés par l'utilisateur `lfs` ne seraient plus lisibles, et GCC échouerait avec `fatal error: stdio.h: Permission denied`.

`LFS=/mnt/lfs` — racine du système LFS

Redéfinie explicitement ici pour deux raisons :

1. Elle l'est déjà dans la session root, mais après `env -i`, toutes les variables ont été effacées. Il faut la redéfinir dans l'environnement propre.
2. Chaque nouveau sous-shell bash lancé pendant la compilation héritera de cette variable (grâce au `export` en bas du fichier), sans avoir besoin de `.bash_profile`.

`LC_ALL=POSIX` — locale neutre forcée

`LC_ALL` est la variable de locale la plus prioritaire : elle écrase toutes les autres (`LANG`, `LC_MESSAGES`, `LC_COLLATE`…). En la forçant à `POSIX` (équivalent de `C`), on obtient trois garanties importantes pour la compilation :

1. **Messages en anglais** : les scripts `configure` analysent les sorties des outils (`gcc -v`, `ld --version`, etc.) avec des expressions régulières en anglais. Si GCC affiche "version 15.2.0" en français ("version 15.2.0" — OK dans ce cas, mais certains messages varient), les regex rate et `configure` prend de mauvaises décisions. Exemple réel : `configure` cherche `"No such file"` dans les messages d'erreur — en français ce serait `"Aucun fichier ou dossier de ce type"`, ce qui ferait échouer la détection.

2. **Ordre de tri ASCII prévisible** : avec une locale UTF-8 française, `sort` peut ordonner les caractères différemment (les lettres accentuées s'intercalent entre les lettres ASCII). Certains tests de compilation supposent l'ordre ASCII strict où `Z` < `a`. Un ordre différent peut faire échouer des tests qui comparent des listes triées.

3. **Pas de conversion de caractères** : en locale POSIX, les chaînes sont traitées comme des suites d'octets, sans interprétation Unicode. Les compilateurs et linkers traitent du code binaire — activer la conversion Unicode peut corrompre silencieusement des données binaires passant par certains filtres de texte dans les Makefiles.

`LFS_TGT=$(uname -m)-lfs-linux-gnu` — le triplet de compilation croisée

C'est la variable la plus importante de tout le processus de construction. Comprendre son rôle est essentiel pour comprendre pourquoi LFS fonctionne.

Qu'est-ce qu'un triplet ?

Un **triplet** (parfois quadruplet) est une chaîne standardisée de la forme `ARCHITECTURE-VENDOR-SYSTÈME` qui identifie complètement une plateforme cible pour la compilation. Il est utilisé par GCC, Binutils et les scripts Autoconf pour savoir pour quelle plateforme produire du code.

Composant	Valeur LFS	Description
`ARCHITECTURE`	`x86_64`	Type de CPU cible (retourné par `uname -m`)
`VENDOR`	`lfs`	Identifiant du "fournisseur" — librement choisi
`SYSTÈME`	`linux-gnu`	Noyau et ABI cibles

Le triplet de l'hôte Debian sur le même matériel serait `x86_64-pc-linux-gnu`. La seule différence est le VENDOR : `pc` vs `lfs`.

Pourquoi changer le VENDOR suffit à activer la compilation croisée ?

GCC considère qu'il fait de la compilation croisée dès que le triplet cible (`--target`) diffère du triplet hôte (`--build`). En changeant `pc` en `lfs`, les deux triplets sont différents — GCC bascule en mode cross-compiler : il cherchera les bibliothèques et headers dans le sysroot (`$LFS`) plutôt que dans les chemins système de l'hôte, et produira des binaires pour la plateforme `x86_64-lfs-linux-gnu`.

Conséquence sur les noms des outils

Quand Binutils est compilé avec `--target=$LFS_TGT`, tous ses outils sont préfixés par le triplet :

- `x86_64-lfs-linux-gnu-as` (assembleur croisé)
- `x86_64-lfs-linux-gnu-ld` (linker croisé)
- `x86_64-lfs-linux-gnu-gcc` (GCC croisé, après sa compilation)

Ce préfixage permet d'avoir les deux versions (hôte et croisée) dans le même `$PATH` sans conflit. Les scripts `configure` qui font `$CC --version` trouvent le bon compilateur car on leur passe `CC=$LFS_TGT-gcc`.

`PATH` — ordre de priorité et évolution au fil des chapitres

```
PATH=/usr/bin
if [ ! -L /bin ]; then PATH=/bin:$PATH; fi
PATH=$LFS/tools/bin:$PATH
```

La construction du `PATH` suit une logique précise en trois niveaux :

Niveau 1 — `/usr/bin` : les outils hôte Debian de base. Nécessaires au départ car `$LFS/tools` est vide. Contient `gcc`, `make`, `tar`, `wget`, etc. de Debian.

Niveau 2 — `/bin` (conditionnel) : sur les systèmes Linux modernes (dont Debian), `/bin` est un lien symbolique vers `/usr/bin` — les deux répertoires sont identiques. La condition `if [ ! -L /bin ]` (si `/bin` n'est *pas* un lien symbolique) ajoute `/bin` au `PATH` uniquement sur les systèmes anciens où `/bin` et `/usr/bin` sont séparés. Sur notre Debian, cette ligne n'a aucun effet.

Niveau 3 — `$LFS/tools/bin` en tête : les outils LFS croisés ont la priorité absolue. Au début du chapitre 5, ce répertoire est vide. Il se remplit progressivement :

- Après Binutils pass 1 → `x86_64-lfs-linux-gnu-ld`, `x86_64-lfs-linux-gnu-as`...
- Après GCC pass 1 → `x86_64-lfs-linux-gnu-gcc`, `x86_64-lfs-linux-gnu-g++`...

Dès qu'un outil LFS existe dans `$LFS/tools/bin`, il prend le dessus sur la version Debian, grâce à sa position en tête de `PATH`.

Évolution du PATH selon les phases :

Phase	PATH effectif	Outils utilisés

Phase	PATH effectif	Outils utilisés
Début chapitre 5	`/mnt/lfs/tools/bin` (vide) → `/usr/bin`	GCC Debian
Après Binutils pass 1	`/mnt/lfs/tools/bin/x86_64-lfs-linux-gnu-*` → `/usr/bin`	Linker LFS + GCC Debian
Après GCC pass 1	`/mnt/lfs/tools/bin/x86_64-lfs-linux-gnu-gcc` → `/usr/bin`	GCC croisé LFS
Chapitre 6	`$LFS/tools/bin` complet → `/usr/bin`	Tous les outils LFS
Dans le chroot (ch.7+)	`/usr/bin:/usr/sbin` (sans `$LFS/tools`)	Outils LFS natifs dans `/usr`

`CONFIG_SITE=$LFS/usr/share/config.site` — réponses pré-calculées pour autoconf

Les scripts `configure` générés par Autoconf détectent les fonctionnalités du système en compilant et en **exécutant** de petits programmes de test. En compilation croisée, cette exécution est impossible : on compile pour `x86_64-lfs-linux-gnu` depuis un hôte `x86_64-pc-linux-gnu` — même si le CPU est le même, le système d'exploitation (au sens des bibliothèques et du sysroot) est différent.

Face à un test qu'il ne peut pas exécuter, `configure` a deux comportements selon les paquets :

- Il assume la valeur la plus conservatrice (souvent "non supporté") → peut désactiver des optimisations
- Il échoue avec une erreur explicite demandant à fournir la valeur manuellement

`config.site` est un fichier shell qui est sourcé par `configure` en début d'exécution. Il peut pré-définir des variables de cache du style :

```
ac_cv_func_malloc_0_nonnull=yes
ac_cv_func_mmap_fixed_mapped=yes
gl_cv_func_working_mkstemp=yes
```

Ces variables court-circuitent les tests correspondants : `configure` les lit dans le cache et saute le test d'exécution. En pointant `CONFIG_SITE` vers `$LFS/usr/share/config.site`, on fournit des réponses adaptées à la plateforme LFS pour tous les scripts `configure` qui en ont besoin.

`export` — rendre les variables disponibles aux processus enfants

La ligne finale du `.bashrc` est essentielle :

```
export LFS LC_ALL LFS_TGT PATH CONFIG_SITE
```

En bash, une variable définie sans `export` est **locale au shell courant** — les processus enfants ne la voient pas. `export` marque une variable comme faisant partie de l'environnement transmis à tout processus lancé depuis ce shell.

Concrètement, quand `make` lance GCC, ou quand un script `configure` lance `gcc`, ces processus enfants héritent des variables exportées. Sans `export LFS_TGT`, le compilateur croisé ne saurait pas quel triplet cibler. Sans `export PATH`, les sous-processus utiliseraient le `PATH` par défaut du système — et trouveraient le GCC Debian au lieu du GCC croisé LFS.

Parallélisme de compilation

```
cat >> ~/.bashrc << "EOF"
export MAKEFLAGS=-j$(nproc)
EOF
```

`MAKEFLAGS` est une variable d'environnement lue par `make` à chaque invocation, comme si les options qu'elle contient avaient été passées en ligne de commande. `-j$(nproc)` lui dit d'utiliser autant de jobs parallèles que de cœurs CPU disponibles.

`$(nproc)` : la commande `nproc` retourne le nombre de processeurs disponibles pour le processus courant. Sur une VM à 4 cœurs, `nproc` retourne 4, et `MAKEFLAGS=-j4` permet à make de lancer 4 compilations simultanées.

Impact sur les temps de compilation :

Paquets	Temps séquentiel (-j1)	Temps parallèle (-j4)	Temps parallèle (-j8)
Binutils	~5 min	~2 min	~1.5 min
GCC pass 1	~40 min	~12 min	~8 min
GCC pass 2 (ch.6)	~55 min	~15 min	~10 min
GCC final (ch.8)	~90 min	~25 min	~15 min
Total LFS	**~8h**	**~2.5h**	**~1.5h**

> **Note :** Le parallélisme peut parfois révéler des bugs dans des Makefiles mal écrits (dépendances implicites entre cibles non déclarées). Si un paquet échoue mystérieusement avec `-j$(nproc)` mais fonctionne avec `-j1`, c'est probablement un bug de Makefile. Dans LFS 13.0, les paquets officiels sont bien testés — si ça arrive, vérifier d'abord qu'il ne manque pas un prérequis.

> **Note sur le SBU :** Dans le livre LFS, les durées sont exprimées en **SBU** (Standard Build Unit). 1 SBU = durée de compilation de Binutils pass 1 sur votre machine. Toutes les autres durées sont relatives à cette référence. C'est une unité relative qui s'adapte à la puissance de la machine utilisée.

Activation et vérification

```
source ~/.bash_profile
```

Cette commande relit `.bash_profile` dans la session actuelle, déclenchant le `exec env -i ... bash` qui repart sur un environnement propre. C'est équivalent à se déconnecter et se reconnecter en tant que `lfs`.

Pour vérifier que l'environnement est correct avant de commencer la compilation :

```
echo $LFS           # Doit afficher : /mnt/lfs
echo $LFS_TGT       # Doit afficher : x86_64-lfs-linux-gnu
echo $LC_ALL        # Doit afficher : POSIX
echo $PATH          # Doit commencer par /mnt/lfs/tools/bin:
echo $MAKEFLAGS     # Doit afficher : -j4 (ou le nombre de cœurs)
echo $CONFIG_SITE   # Doit afficher : /mnt/lfs/usr/share/config.site
```

Si une variable est vide ou incorrecte, ne pas continuer — diagnostiquer le problème avant de lancer quoi que ce soit.

Vérification que le montage est toujours actif :

```
mountpoint -q $LFS && echo "OK : $LFS est monté" || echo "ERREUR : $LFS
n'est pas monté !"
```

> **Attention :** Toujours utiliser `su - lfs` (avec le tiret) pour se connecter en tant que lfs. Sans le tiret (`su lfs`), bash ne source pas `.bash_profile` et hérite de l'environnement pollué de la session root. C'est l'une des erreurs les plus fréquentes et les plus sournoises du processus LFS — tout semble fonctionner, mais les mauvais outils sont utilisés en silence.

Résumé des fichiers créés et de leur rôle :

Fichier	Lu quand	Rôle
`~/.bash_profile`	Connexion login (`su - lfs`)	Nettoie l'environnement via `exec env -i`, lance bash propre
`~/.bashrc`	Chaque shell interactif non-login	Définit et exporte toutes les variables de compilation

---

7. Compilation croisée — Phase 1

Concept de la compilation croisée

La compilation croisée consiste à utiliser un compilateur sur la machine A (l'hôte Debian) pour produire des binaires qui s'exécuteront sur la machine B (le futur système LFS). Dans notre cas A et B sont physiquement la même machine, mais le compilateur croisé traite LFS comme une plateforme distincte grâce au triplet `$LFS_TGT`.

LFS suit une stratégie en **deux passes** pour s'assurer que le système final est 100% indépendant de l'hôte :

- **Passe 1** (chapitre 5) : construire une chaîne d'outils croisée minimale dans `$LFS/tools`. Ces outils sont compilés *par* le compilateur hôte et *pour* la plateforme LFS.
- **Passe 2** (chapitre 6) : recompiler ces mêmes outils en utilisant la chaîne croisée de la passe 1. Le résultat ne contient plus aucune référence à l'hôte.

Cette double compilation est la garantie que le système LFS est entièrement auto-suffisant.

7.1 Binutils (Pass 1)

Binutils est compilé en premier car il fournit l'**assembleur** et le **linker** — les outils les plus fondamentaux de la chaîne de compilation. GCC ne peut pas être compilé sans eux, et ils doivent impérativement cibler la bonne plateforme avant que GCC soit construit.

(~1 SBU de référence)

```
tar -xf binutils-2.46.0.tar.xz && cd binutils-2.46.0
mkdir build && cd build

../configure                        \
    --prefix=$LFS/tools             \
    --with-sysroot=$LFS             \
    --target=$LFS_TGT               \
    --disable-nls                   \
    --enable-gprofng=no             \
    --disable-werror                \
    --enable-new-dtags              \
    --enable-default-hash-style=gnu

make
make install
```

Option	Raison
`--with-sysroot=$LFS`	Indique au linker de chercher les bibliothèques dans `$LFS` plutôt que sur l'hôte
`--target=$LFS_TGT`	Génère des outils ciblant la plateforme LFS (compilation croisée)
`--disable-nls`	Pas besoin d'internationalisation pour les outils de build temporaires
`--enable-gprofng=no`	Évite une dépendance à Java non nécessaire à ce stade
`--enable-new-dtags`	Utilise `RUNPATH` plutôt que l'ancien `RPATH` pour les chemins de bibliothèques dynamiques

> **Vérification :** Après installation, `$LFS/tools/bin/x86_64-lfs-linux-gnu-ld` doit exister. C'est l'assembleur croisé.

7.2 GCC (Pass 1)

GCC est le compilateur C/C++. À ce stade, on le compile de façon **intentionnellement limitée** : sans bibliothèque C (Glibc n'existe pas encore), sans support des threads, sans bibliothèques de runtime. Son seul rôle dans cette passe est de pouvoir compiler Glibc et Libstdc++ à l'étape suivante.

(~11 SBU)

GCC nécessite trois bibliothèques mathématiques : **GMP** (arithmétique entière), **MPFR** (flottant multi-précision) et **MPC** (nombres complexes). Elles sont extraites directement dans l'arborescence source de GCC pour être compilées avec lui.

```
tar -xf gcc-15.2.0.tar.xz && cd gcc-15.2.0

# Sur x86_64, forcer l'utilisation de 'lib' au lieu de 'lib64' — LFS
n'utilise pas lib64
case $(uname -m) in
  x86_64) sed -e '/m64=/s/lib64/lib/' -i.orig gcc/config/i386/t-linux64 ;;
esac

tar -xf ../mpfr-4.2.2.tar.xz && mv -v mpfr-4.2.2 mpfr
tar -xf ../gmp-6.3.0.tar.xz  && mv -v gmp-6.3.0  gmp
tar -xf ../mpc-1.3.1.tar.gz  && mv -v mpc-1.3.1  mpc

mkdir build && cd build

../configure                  \
    --target=$LFS_TGT         \
    --prefix=$LFS/tools       \
    --with-glibc-version=2.43 \
    --with-sysroot=$LFS       \
    --with-newlib             \
    --without-headers         \
    --enable-default-pie      \
    --enable-default-ssp      \
    --disable-nls             \
    --disable-shared          \
    --disable-multilib        \
    --disable-threads         \
    --disable-libatomic       \
    --disable-libgomp         \
    --disable-libquadmath     \
    --disable-libssp          \
    --disable-libvtv          \
    --disable-libstdcxx       \
    --enable-languages=c,c++

make && make install
```

Après l'installation, on génère un fichier `limits.h` qui définit les limites numériques de la plateforme cible. Sans ce fichier, certains paquets compilés avec ce GCC croisé pourraient utiliser des limites incorrectes.

```
cd ..
cat gcc/limitx.h gcc/glimits.h gcc/limity.h > \
  $(dirname $($LFS_TGT-gcc -print-libgcc-file-name))/include/limits.h
```

Option	Raison
`--with-newlib`	Utilise newlib comme stub de bibliothèque C — Glibc n'est pas encore disponible
`--without-headers`	N'utilise pas les headers du système hôte, évite toute contamination
`--disable-shared`	Compile GCC entièrement en statique — pas de dépendance aux `.so` de l'hôte
`--disable-multilib`	Pas de support 32 bits — LFS est purement 64 bits
`--disable-libstdcxx`	Libstdc++ sera compilée séparément (§7.5) après Glibc

> **Attention :** Ce GCC pass 1 ne peut pas encore compiler de programmes complets. Il manque la bibliothèque C. Son unique rôle est de servir de tremplin pour compiler Glibc.

7.3 Linux API Headers

Avant de compiler Glibc, il faut installer les **headers du noyau Linux**. Ces fichiers `.h` définissent l'interface entre les programmes en espace utilisateur et le noyau : numéros d'appels système, structures de données des sockets, constantes d'`ioctl`, etc. Glibc en a besoin pour savoir comment communiquer avec le noyau au moment de la compilation.

```
tar -xf linux-6.18.10.tar.xz && cd linux-6.18.10

make mrproper   # Nettoie les fichiers générés pré-existants dans l'archive
make headers
find usr/include -type f ! -name '*.h' -delete   # Garder uniquement les .h
cp -rv usr/include $LFS/usr
```

> **Note :** `make mrproper` est obligatoire même sur une archive fraîche — certains tarballs du kernel contiennent des fichiers `.config` résiduels qui pourraient corrompre l'extraction des headers.

7.4 Glibc

(~4.6 SBU)

Glibc est la **bibliothèque C standard** — le composant le plus fondamental de tout système Linux. Elle implémente les fonctions C standard (`printf`, `malloc`, `open`, `fork`…) et fait le lien entre les programmes et le noyau. Pratiquement tous les programmes compilés pour Linux en dépendent. Sans Glibc, rien ne peut fonctionner.

```
tar -xf glibc-2.43.tar.xz && cd glibc-2.43

# Liens symboliques pour le chargeur dynamique (ld-linux)
case $(uname -m) in
    i?86)   ln -sfv ld-linux.so.2 $LFS/lib/ld-lsb.so.3 ;;
    x86_64) ln -sfv ../lib/ld-linux-x86-64.so.2 $LFS/lib64
            ln -sfv ../lib/ld-linux-x86-64.so.2 $LFS/lib64/ld-lsb-x86-
64.so.3 ;;
esac

patch -Np1 -i ../glibc-fhs-1.patch   # Conformité FHS : place ldconfig dans
/usr/sbin

mkdir -v build && cd build
echo "rootsbindir=/usr/sbin" > configparms

../configure                             \
      --prefix=/usr                      \
      --host=$LFS_TGT                    \
      --build=$(../scripts/config.guess) \
      --disable-nscd                     \
      libc_cv_slibdir=/usr/lib           \
      --enable-kernel=5.4

make
make DESTDIR=$LFS install
sed '/RTLDLIST=/s@/usr@@g' -i $LFS/usr/bin/ldd
```

> **Note normale :** Un avertissement sur `msgfmt` manquant apparaît — c'est attendu et sans conséquence. `msgfmt` fait partie de Gettext, installé plus tard.

> **Vérification critique :** Après l'installation, tester que le chargeur dynamique est correctement en place :
>
> ```
> echo 'int main(){}' | $LFS_TGT-gcc -x c -
> readelf -l a.out | grep ld-linux
> ```
>
> Doit afficher `/lib64/ld-linux-x86-64.so.2`. Si le chemin pointe vers l'hôte, la compilation croisée est corrompue.

7.5 Libstdc++

(~0.4 SBU)

Libstdc++ est la **bibliothèque C++ standard** (équivalente à Glibc, mais pour le C++). Elle est compilée depuis les sources de GCC mais séparément du compilateur lui-même, car elle nécessite Glibc (installée juste avant) pour fonctionner. Tous les programmes C++ en dépendront.

```
cd gcc-15.2.0   # Répertoire GCC déjà extrait lors du pass 1
mkdir -v build && cd build

../libstdc++-v3/configure       \
    --host=$LFS_TGT             \
    --build=$(../config.guess)  \
    --prefix=/usr               \
    --disable-multilib          \
    --disable-nls               \
    --disable-libstdcxx-pch     \
    --with-gxx-include-dir=/tools/$LFS_TGT/include/c++/15.2.0

make
make DESTDIR=$LFS install
rm -v $LFS/usr/lib/lib{stdc++{,exp,fs},supc++}.la
```

Les fichiers `.la` supprimés à la fin sont des métadonnées libtool qui, en contexte de compilation croisée, contiennent des chemins absolus vers l'hôte. Les laisser en place ferait que les paquets suivants pourraient accidentellement lier contre des bibliothèques de l'hôte au lieu de celles de LFS.

---

8. Outils temporaires croisés — Chapitre 6

Vue d'ensemble

À la fin du chapitre 5, on dispose d'une Glibc et d'un GCC croisés dans `$LFS`. C'est nécessaire mais insuffisant : le chroot qui sera créé au chapitre 7 a besoin d'un **ensemble complet d'outils** pour fonctionner — un shell, des utilitaires de base, des outils de compression, un `make`... Sans eux, impossible de lancer la moindre commande dans l'environnement isolé.

Le chapitre 6 construit ces 17 paquets en **compilation croisée** : ils sont compilés par le compilateur hôte mais avec `--host=$LFS_TGT` pour cibler la plateforme LFS et s'installer avec `DESTDIR=$LFS`. À la fin, `$LFS/usr/` contient un système rudimentaire mais opérationnel.

Pourquoi ne pas les compiler directement dans le chroot ? Parce que le chroot n'est pas encore entré. Le chapitre 6 prépare le terrain pour que l'entrée dans le chroot (chapitre 7) soit possible. C'est une étape de bootstrap.

L'ordre des paquets n'est pas arbitraire : chaque paquet peut dépendre de ceux qui le précèdent. M4 avant Ncurses, Ncurses avant Bash, Bash avant tout ce qui exécute des scripts configure, etc.

> **Rappel :** Tous ces paquets doivent être compilés en tant qu'utilisateur `lfs` avec l'environnement du chapitre 6 actif (`source ~/.bash_profile`).

8.1 M4

Rôle : M4 est un processeur de macros généraliste. Il lit un fichier texte, remplace les appels de macros par leur définition, et produit un texte transformé en sortie. Son utilisation la plus connue est comme moteur

interne d'**Autoconf** : tous les scripts `configure` que l'on exécutera dans les chapitres suivants sont générés par Autoconf, qui utilise M4 pour les produire.

Pourquoi maintenant : M4 est l'une des premières dépendances de la chaîne de build. Il n'a lui-même aucune dépendance particulière, ce qui en fait un bon point de départ. Il doit être présent dans `$LFS` avant que l'on entre dans le chroot, car certains paquets du chapitre 7 l'utilisent indirectement via Autoconf.

```
./configure --prefix=/usr    \
            --host=$LFS_TGT  \
            --build=$(build-aux/config.guess)
make
make DESTDIR=$LFS install
```

La configuration est simple : pas d'options particulières. `--host=$LFS_TGT` est la clé — elle dit au système de build que le binaire produit devra s'exécuter sur la plateforme LFS, pas sur l'hôte.

8.2 Ncurses

Rôle : Ncurses (New Curses) est une bibliothèque qui permet aux programmes de contrôler précisément l'affichage dans un terminal texte : positionner le curseur, dessiner des cadres, gérer les couleurs, réagir aux touches spéciales. Elle est indispensable pour des programmes comme **Bash** (complétion interactive), **Less** (défilement), **Vim** (interface entière), **htop**, et de nombreux installeurs.

Pourquoi avant Bash : Bash est compilé avec le support de readline, qui lui-même dépend de Ncurses pour gérer le terminal. L'ordre est donc : Ncurses → Bash.

La subtilité du `tic` : Ncurses maintient une base de données de terminaux (les fichiers dans `/usr/share/terminfo`) qui décrit les capacités de chaque type de terminal. L'outil `tic` compile les descriptions de terminaux en format binaire. Mais en compilation croisée, on ne peut pas exécuter les binaires cibles directement — il faut donc d'abord compiler un `tic` natif (qui s'exécute sur l'hôte) pour que l'installation puisse traiter la base de données.

```
# Étape 1 : compiler 'tic' pour l'hôte (sera exécuté pendant
l'installation)
mkdir build && pushd build
  ../configure --prefix=$LFS/tools AWK=gawk
  make -C include
  make -C progs tic
  install progs/tic $LFS/tools/bin
popd

# Étape 2 : compiler Ncurses pour LFS
./configure --prefix=/usr                    \
            --host=$LFS_TGT                  \
            --build=$(./config.guess)        \
            --mandir=/usr/share/man          \
            --with-manpage-format=normal     \
            --with-shared                    \
```

```
              --without-normal                  \
              --with-cxx-shared                 \
              --without-debug                   \
              --without-ada                     \
              --disable-stripping               \
              AWK=gawk
  make
  make DESTDIR=$LFS install

  ln -sv libncursesw.so $LFS/usr/lib/libncurses.so
  sed -e 's/^#if.*XOPEN.*$/#if 1/' -i $LFS/usr/include/curses.h
```

Option	Raison
`--with-shared`	Produit des bibliothèques dynamiques (`.so`) plutôt que statiques
`--without-normal`	N'installe pas les versions statiques — inutiles dans ce contexte
`--with-cxx-shared`	La version C++ de Ncurses aussi en dynamique
`--without-ada`	Pas de bindings Ada — le compilateur Ada n'est pas disponible
`AWK=gawk`	Force l'utilisation de gawk plutôt que mawk (par défaut sur Debian) qui est moins compatible

Le lien symbolique `libncurses.so → libncursesw.so` est nécessaire car la version "wide" (`ncursesw`, qui supporte Unicode) doit aussi être accessible sous le nom standard `ncurses` pour les programmes qui ne spécifient pas le suffixe `w`.

8.3 Bash

Rôle : Bash est le shell qui sera utilisé dans le chroot. Sans lui, impossible d'exécuter le moindre script, de taper des commandes interactives, ou de faire fonctionner les scripts `configure` des paquets suivants. C'est l'interpréteur de commandes central du système en construction.

Pourquoi maintenant : Bash doit être installé avant d'entrer dans le chroot, car c'est lui qui sera invoqué par la commande `chroot` elle-même. Sans Bash dans `$LFS`, la commande `chroot "$LFS" /bin/bash` échouerait immédiatement.

```
  ./configure --prefix=/usr                       \
              --build=$(sh support/config.guess) \
              --host=$LFS_TGT                     \
              --without-bash-malloc
  make
  make DESTDIR=$LFS install

  ln -sv bash $LFS/bin/sh
```

`--without-bash-malloc` mérite une explication : Bash embarque son propre allocateur mémoire qui, dans certains environnements de cross-compilation ou avec certaines versions de Glibc, peut provoquer des segmentation faults. En désactivant cette option, Bash utilise `malloc()` directement depuis Glibc, ce qui est plus stable.

Le lien symbolique `sh → bash` est important : de nombreux scripts système utilisent `#!/bin/sh` comme shebang. Sur LFS, `/bin/sh` doit pointer vers Bash pour que ces scripts bénéficient des fonctionnalités étendues de Bash.

8.4 Coreutils

Rôle : Coreutils regroupe les utilitaires les plus fondamentaux du système Unix : `ls`, `cp`, `mv`, `rm`, `mkdir`, `chmod`, `chown`, `cat`, `echo`, `sort`, `head`, `tail`, `wc`, `cut`, `date`, `hostname`... Pratiquement toutes les opérations de base sur les fichiers et le système passent par ces outils. Ils sont nécessaires dès l'entrée dans le chroot.

Pourquoi maintenant : Le chroot lui-même, une fois entré, exécutera immédiatement des commandes comme `mkdir`, `cp`, `ln` pour créer la structure FHS. Ces commandes viennent de Coreutils. Sans eux dans `$LFS`, le chroot serait inutilisable.

```
./configure --prefix=/usr                      \
            --host=$LFS_TGT                    \
            --build=$(build-aux/config.guess) \
            --enable-install-program=hostname \
            --enable-no-install-program=kill,uptime
make
make DESTDIR=$LFS install

mv -v $LFS/usr/bin/chroot              $LFS/usr/sbin
mkdir -pv $LFS/usr/share/man/man8
mv -v $LFS/usr/share/man/man1/chroot.1 $LFS/usr/share/man/man8/chroot.8
sed -i 's/"1"/"8"/'                    $LFS/usr/share/man/man8/chroot.8
```

`--enable-install-program=hostname` : Par défaut, Coreutils n'installe pas `hostname` car d'autres paquets le fournissent aussi (comme `inetutils`). On le force ici car `hostname` est nécessaire tôt dans le processus. `--enable-no-install-program=kill,uptime` exclut ces deux commandes qui seront fournies par d'autres paquets (`procps-ng` pour `kill`).

Le déplacement de `chroot` dans `/usr/sbin` et sa page de man en section 8 suit la convention FHS : les commandes d'administration système destinées au root vont dans `sbin`, et leurs pages de man en section 8.

8.5 Diffutils

Rôle : Diffutils fournit les outils `diff` (compare deux fichiers ligne par ligne) et `cmp` (compare octet par octet). Ils sont utilisés partout dans les systèmes de build pour détecter des changements, appliquer des patches, et dans certaines suites de tests pour comparer des sorties attendues vs réelles.

Particularité de compilation croisée : Le script `configure` de Diffutils tente d'exécuter un petit programme de test pour vérifier que `strcasecmp` fonctionne correctement. En compilation croisée, ce test ne peut pas s'exécuter (on ne peut pas lancer des binaires cibles sur l'hôte). On court-circuite ce test en fournissant la réponse directement.

```
./configure --prefix=/usr --host=$LFS_TGT \
            gl_cv_func_strcasecmp_works=y  \
            --build=$(./build-aux/config.guess)
make
make DESTDIR=$LFS install
```

`gl_cv_func_strcasecmp_works=y` : Cette variable de cache dit explicitement à `configure` que `strcasecmp` fonctionne, évitant le test d'exécution. Sans cette variable, `configure` échouerait avec une erreur de cross-compilation.

8.6 File

Rôle : La commande `file` identifie le type d'un fichier en analysant son contenu (les "magic bytes" au début du fichier) plutôt que son extension. Elle retourne des descriptions comme `ELF 64-bit LSB executable`, `ASCII text`, `gzip compressed data`, etc. Elle est utilisée par de nombreux scripts de build pour décider comment traiter un fichier.

La double compilation : Comme pour Ncurses et `tic`, le problème est que `file` utilise son propre exécutable *pendant sa compilation* pour traiter les fichiers de données de magic. En cross-compilation, on ne peut pas exécuter le binaire cible, donc on compile d'abord une version native pour l'hôte.

```
# Étape 1 : version native pour l'hôte, utilisée pendant la compilation
mkdir build && pushd build
  ../configure --disable-bzlib --disable-libseccomp \
               --disable-xzlib --disable-zlib
  make
popd

# Étape 2 : version pour LFS, utilise le 'file' natif compilé ci-dessus
./configure --prefix=/usr --host=$LFS_TGT --build=$(./config.guess)
make FILE_COMPILE=$(pwd)/build/src/file
make DESTDIR=$LFS install
rm -v $LFS/usr/lib/libmagic.la
```

Les `--disable-*` de l'étape 1 évitent que la version native cherche des bibliothèques de compression (bzip2, xz, zlib) qui ne sont peut-être pas disponibles avec les bons headers sur l'hôte. Pour l'usage temporaire qu'on en fait, ces fonctionnalités sont inutiles.

8.7 Findutils

Rôle : Findutils fournit `find` (recherche de fichiers selon des critères variés : nom, taille, date, permissions…) et `xargs` (exécute une commande avec une liste d'arguments lus sur l'entrée standard). Ces outils sont omniprésents dans les scripts de build et d'administration système.

```
./configure --prefix=/usr --host=$LFS_TGT \
            --build=$(build-aux/config.guess)
make
make DESTDIR=$LFS install
```

Pas de particularité notable. La compilation est standard. `find` et `xargs` seront indispensables dès l'entrée dans le chroot, notamment pour les opérations de nettoyage et de recherche dans les arborescences de paquets.

8.8 Gawk

Rôle : Gawk est l'implémentation GNU de AWK, un langage de traitement de texte orienté lignes et colonnes. AWK est massivement utilisé dans les scripts de build pour extraire des informations de fichiers texte (versions, chemins, résultats de tests). C'est l'une des raisons pour lesquelles on s'est assuré que `/usr/bin/awk` pointe vers gawk sur l'hôte dès le chapitre 2.

```
sed -i 's/extras//' Makefile.in   # Supprimer une cible de build non
nécessaire
./configure --prefix=/usr --host=$LFS_TGT \
            --build=$(build-aux/config.guess)
make
make DESTDIR=$LFS install
```

La ligne `sed -i 's/extras//'` supprime une cible `extras` dans le Makefile qui essaierait d'installer des extensions Gawk non nécessaires dans `$LFS` et dont la compilation peut échouer en contexte croisé.

8.9 Grep

Rôle : `grep` recherche des patterns (expressions régulières) dans des fichiers ou sur l'entrée standard. Avec `find` et `sed`, c'est l'un des outils de traitement de texte les plus utilisés dans les scripts Unix. Pratiquement tous les scripts `configure` utilisent `grep` pour détecter des fonctionnalités du système.

```
./configure --prefix=/usr --host=$LFS_TGT \
            --build=$(./build-aux/config.guess)
make
make DESTDIR=$LFS install
```

Compilation standard. La version de grep installée dans `$LFS` remplacera le grep de l'hôte une fois dans le chroot, garantissant que les scripts configure du chapitre 8 utilisent la version LFS plutôt que celle du

système Debian.

8.10 Gzip

Rôle : `gzip` est l'outil de compression/décompression au format `.gz`. Il est nécessaire pour extraire de nombreuses archives de paquets (`.tar.gz`) et est utilisé par `zcat`, `zgrep` et d'autres outils de la bibliothèque standard. C'est aussi le format par défaut des pages de man compressées.

```
./configure --prefix=/usr --host=$LFS_TGT \
            --build=$(./build-aux/config.guess)
make
make DESTDIR=$LFS install
```

Compilation standard. À noter que gzip sera complété par d'autres outils de compression dans le chapitre 8 : bzip2, xz, lz4, zstd. LFS utilise plusieurs formats car certains paquets sont distribués dans des formats spécifiques.

8.11 Make

Rôle : `make` est le moteur de build standard. Il lit un `Makefile`, détermine quels fichiers sont à jour et lesquels doivent être recompilés, et exécute les règles dans le bon ordre. Pratiquement tous les paquets LFS utilisent `make` pour leur compilation. Sans `make` dans le chroot, aucun paquet du chapitre 8 ne pourrait être compilé.

```
./configure --prefix=/usr --host=$LFS_TGT \
            --build=$(build-aux/config.guess) \
            --without-guile
make
make DESTDIR=$LFS install
```

`--without-guile` évite que Make cherche à intégrer un interpréteur Scheme (Guile) qui n'est pas disponible dans l'environnement de compilation croisée et n'est pas nécessaire pour LFS.

8.12 Patch

Rôle : `patch` applique des fichiers diff (produits par `diff -u`) à du code source. LFS utilise plusieurs patches officiels pour corriger des bugs, adapter du code à des versions récentes de GCC, ou assurer la conformité FHS. Sans `patch` disponible dans `$LFS`, ces corrections ne pourraient pas être appliquées lors du chapitre 8.

```
./configure --prefix=/usr --host=$LFS_TGT \
            --build=$(build-aux/config.guess)
```

```
make
make DESTDIR=$LFS install
```

Compilation standard. Les 5 patches utilisés dans LFS 13.0 sont :

- `bzip2-install_docs` : corrige l'installation de la documentation
- `coreutils-i18n` : ajoute le support international à certains outils
- `expect-gcc15` : adapte Expect à GCC 15
- `glibc-fhs` : assure la conformité FHS de Glibc
- `kbd-backspace` : corrige le comportement de la touche Backspace

8.13 Sed

Rôle : `sed` (stream editor) est un éditeur de texte non-interactif qui transforme un flux de texte en appliquant des expressions de substitution, suppression, insertion. C'est l'outil de manipulation de texte le plus utilisé dans les scripts de build LFS, notamment pour patcher des Makefiles, corriger des chemins dans des scripts, ou adapter des fichiers de configuration.

```
./configure --prefix=/usr --host=$LFS_TGT \
            --build=$(./build-aux/config.guess)
make
make DESTDIR=$LFS install
```

Compilation standard. Sed est utilisé des dizaines de fois dans les scripts de build du chapitre 8 pour des transformations comme `sed -i 's/lib64/lib/'` (correction des chemins 64 bits), `sed -i '/test/d'` (suppression de lignes dans des Makefiles), etc.

8.14 Tar

Rôle : `tar` (tape archive) crée et extrait des archives `.tar`, qui sont le format de distribution universel des paquets source sur Unix/Linux. Chaque paquet du chapitre 8 arrive sous forme d'archive `.tar.xz`, `.tar.gz`, ou `.tar.bz2`. Sans `tar` dans le chroot, impossible d'extraire quoi que ce soit.

```
./configure --prefix=/usr --host=$LFS_TGT \
            --build=$(build-aux/config.guess) \
            FORCE_UNSAFE_CONFIGURE=1
make
make DESTDIR=$LFS install
```

`FORCE_UNSAFE_CONFIGURE=1` est nécessaire parce que le script configure de Tar détecte qu'il est lancé en tant que root (ce qui n'est pas vrai — on est l'utilisateur `lfs` — mais la détection est parfois faussée en cross-compilation) et refuse de continuer sans ce drapeau. C'est uniquement pour contourner cette vérification, pas une option de comportement.

8.15 Xz

Rôle : `xz` est l'outil de compression au format LZMA/XZ. C'est le format de compression le plus courant pour les sources LFS (la plupart des archives sont en `.tar.xz`). Il offre un meilleur taux de compression que gzip ou bzip2, au prix d'une compression plus lente. La décompression reste rapide.

```
./configure --prefix=/usr --host=$LFS_TGT \
            --build=$(build-aux/config.guess) \
            --disable-static \
            --docdir=/usr/share/doc/xz-5.8.2
make
make DESTDIR=$LFS install
rm -v $LFS/usr/lib/liblzma.la
```

`--disable-static` évite de produire la bibliothèque statique `liblzma.a`, qui n'est pas nécessaire ici et occupe de la place. La suppression du fichier `.la` suit la même logique que pour Libstdc++ : les métadonnées libtool en cross-compilation contiennent des chemins vers l'hôte qui deviendraient incorrects une fois dans le chroot.

8.16 Binutils pass 2

Rôle et contexte : Il s'agit d'une **deuxième compilation** de Binutils, mais cette fois avec des objectifs différents. Le Binutils pass 1 était un outil purement croisé, installé dans `$LFS/tools` et ciblant `$LFS_TGT`. Le Binutils pass 2 est une version *native* : compilé par le cross-compiler de la passe 1, installé dans `$LFS/usr`, il sera l'assembleur et le linker définitifs du système LFS.

La différence fondamentale dans les options de configuration reflète ce changement de rôle : `--prefix=/usr` au lieu de `$LFS/tools`, et l'absence de `--target` (il cible la machine native par défaut).

(~0.4 SBU)

```
# Corriger un bug libtool qui peut linker contre des bibliothèques de
l'hôte
sed '6031s/$add_dir//' -i ltmain.sh

mkdir -v build && cd build

../configure                    \
    --prefix=/usr               \
    --build=$(../config.guess)  \
    --host=$LFS_TGT             \
    --disable-nls               \
    --enable-shared             \
    --enable-gprofng=no         \
    --disable-werror            \
    --enable-64-bit-bfd         \
    --enable-new-dtags          \
    --enable-default-hash-style=gnu
```

```
make
make DESTDIR=$LFS install
rm -v $LFS/usr/lib/lib{bfd,ctf,ctf-nobfd,opcodes,sframe}.{a,la}
```

Le `sed` sur `ltmain.sh` corrige un problème connu de libtool : sans ce patch, libtool peut ajouter des chemins de bibliothèques de l'hôte dans les binaires LFS, provoquant des erreurs subtiles plus tard quand ces chemins n'existent plus dans le chroot.

`--enable-shared` permet à `libbfd` (la bibliothèque interne de Binutils) d'être partagée dynamiquement, réduisant la taille des binaires qui en dépendent. `--enable-64-bit-bfd` assure le support complet des objets 64 bits.

8.17 GCC pass 2

Rôle et contexte : C'est le **compilateur définitif** du système LFS. Contrairement au GCC pass 1 (limité, sans threads, sans bibliothèques runtime), ce GCC pass 2 est complet : il inclut libgcc, libstdc++, le support des threads POSIX, PIE et SSP activés par défaut. Tous les 83 paquets du chapitre 8 seront compilés par ce GCC.

C'est le paquet le plus long de tout le chapitre 6.

(~14 SBU)

```
tar -xf gcc-15.2.0.tar.xz && cd gcc-15.2.0

# Mêmes dépendances GMP/MPFR/MPC qu'au pass 1
tar -xf ../mpfr-4.2.2.tar.xz && mv mpfr-4.2.2 mpfr
tar -xf ../gmp-6.3.0.tar.xz  && mv gmp-6.3.0  gmp
tar -xf ../mpc-1.3.1.tar.gz  && mv mpc-1.3.1  mpc

# Patch x86_64 identique au pass 1
case $(uname -m) in
  x86_64) sed -e '/m64=/s/lib64/lib/' -i.orig gcc/config/i386/t-linux64 ;;
esac

# Activer le support POSIX threads dans libgcc et libstdc++
# (impossible au pass 1 car Glibc n'était pas encore disponible)
sed '/thread_header =/s/@.*@/gthr-posix.h/' \
    -i libgcc/Makefile.in libstdc++-v3/include/Makefile.in

mkdir -v build && cd build

../configure                          \
    --build=$(../config.guess)        \
    --host=$LFS_TGT                   \
    --target=$LFS_TGT                 \
    --prefix=/usr                     \
    --with-build-sysroot=$LFS         \
    --enable-default-pie              \
```

```
      --enable-default-ssp              \
      --disable-nls                     \
      --disable-multilib                \
      --disable-libatomic               \
      --disable-libgomp                 \
      --disable-libquadmath             \
      --disable-libsanitizer            \
      --disable-libssp                  \
      --disable-libvtv                  \
      --enable-languages=c,c++          \
      LDFLAGS_FOR_TARGET=-L$PWD/$LFS_TGT/libgcc

  make
  make DESTDIR=$LFS install

  ln -sv gcc $LFS/usr/bin/cc
```

Option	Raison
`--host=$LFS_TGT` et `--target=$LFS_TGT`	Ce GCC est compilé par la chaîne croisée et produit du code pour LFS — même plateforme pour les deux
`--with-build-sysroot=$LFS`	Indique aux outils de build auxiliaires (scripts, tests) où trouver les fichiers dans `$LFS`
`--enable-default-pie`	Active Position Independent Executables par défaut — meilleure sécurité (ASLR)
`--enable-default-ssp`	Active Stack Smashing Protection par défaut — protection contre les buffer overflows
`--disable-libsanitizer`	Les bibliothèques AddressSanitizer/MemorySanitizer ne sont pas nécessaires dans LFS
`LDFLAGS_FOR_TARGET=...`	Assure que libstdc++ utilise le libgcc de *ce* pass, pas celui du pass 1

Le `sed` sur `gthr-posix.h` active le support des threads POSIX dans libgcc et libstdc++. Au pass 1, ce support était désactivé car Glibc (qui fournit `pthread`) n'existait pas encore. Maintenant qu'elle est installée dans `$LFS`, le support threads peut être activé.

Le lien `cc → gcc` suit la convention UNIX selon laquelle `cc` (C Compiler) est le nom canonique du compilateur C. De nombreux scripts de build appellent `cc` plutôt que `gcc`.

> **Vérification recommandée :** Après GCC pass 2, s'assurer que la chaîne est opérationnelle :
>
> ```
> echo 'int main(){return 0;}' | $LFS_TGT-gcc -x c - -o /tmp/test
> readelf -l /tmp/test | grep interpreter
> ```
>
> Doit afficher `/lib64/ld-linux-x86-64.so.2` — le chargeur dynamique de LFS, pas celui de l'hôte.

9. Entrée dans le Chroot — Chapitre 7

Qu'est-ce qu'un chroot ?

`chroot` (change root) est un appel système qui modifie la perception qu'a un processus de la racine du système de fichiers. Après `chroot /mnt/lfs`, le processus voit `/mnt/lfs` comme son `/`. Il ne peut plus accéder à rien en dehors — l'hôte Debian devient invisible. C'est l'isolation qui permet de construire et configurer le système LFS comme s'il tournait sur sa propre machine.

À partir du chroot, la variable `$LFS` n'est plus nécessaire : le système de fichiers LFS *est* la racine. Toutes les commandes opèrent directement sur `/usr`, `/etc`, etc. — qui sont en réalité `$LFS/usr`, `$LFS/etc` vus depuis l'hôte.

9.1 Changement de propriétaire

Les fichiers compilés au chapitre 5 et 6 appartiennent à l'utilisateur `lfs`, qui n'existera pas dans le système final. On les transfère à `root` (uid 0) avant d'entrer dans le chroot.

```
chown -R root:root $LFS/{usr,var,etc,tools,sources}
case $(uname -m) in
  x86_64) chown -R root:root $LFS/lib64 ;;
esac
```

9.2 Montages des systèmes de fichiers virtuels

Le noyau Linux expose des informations et des interfaces à travers des systèmes de fichiers virtuels qui n'existent pas sur le disque. Sans eux, le chroot serait aveugle : il ne pourrait pas connaître les processus en cours (`/proc`), les périphériques disponibles (`/dev`), la configuration du noyau (`/sys`), ou créer des fichiers temporaires (`/run`). Ces montages font le pont entre le noyau hôte et l'environnement chroot.

```
mkdir -pv $LFS/{dev,proc,sys,run}

mount -v --bind /dev $LFS/dev          # Périphériques (disques,
terminaux...)
mount -vt devpts devpts -o gid=5,mode=0620 $LFS/dev/pts  # Pseudo-terminaux
mount -vt proc   proc   $LFS/proc      # Informations sur les processus
mount -vt sysfs  sysfs  $LFS/sys       # Informations sur le matériel/noyau
mount -vt tmpfs  tmpfs  $LFS/run       # Données volatiles runtime

if [ -h $LFS/dev/shm ]; then
  install -v -d -m 1777 $LFS$(realpath /dev/shm)
else
  mount -vt tmpfs -o nosuid,nodev tmpfs $LFS/dev/shm
fi
```

> **Rappel :** Ces montages sont perdus au redémarrage. Il faut les refaire à chaque reprise avant d'entrer dans le chroot. Le script `04_chroot_prep.sh mount` automatise cette étape.

9.3 Entrée dans le chroot

```
chroot "$LFS" /usr/bin/env -i   \
    HOME=/root                  \
    TERM="$TERM"                \
    PS1='(lfs chroot) \u:\w\$ ' \
    PATH=/usr/bin:/usr/sbin     \
    MAKEFLAGS="-j$(nproc)"      \
    TESTSUITEFLAGS="-j$(nproc)" \
    /bin/bash --login
```

Comme pour `su - lfs`, on utilise `env -i` pour démarrer avec un environnement propre. Le `PATH` ne pointe plus vers `$LFS/tools` — les outils temporaires du chapitre 6 sont maintenant les outils du système.

> **Note :** Le prompt affichera `I have no name!` jusqu'à la création de `/etc/passwd` — c'est normal, le shell ne peut pas encore résoudre l'uid 0 en nom d'utilisateur.

> **Danger :** Les scripts `05_chroot_tools.sh` à `08_kernel_boot.sh` doivent être exécutés **uniquement depuis l'intérieur du chroot**. Les lancer depuis l'hôte modifierait `/usr`, `/etc`, `/lib` directement sur le système Debian.

9.4 Création de la structure FHS

```
mkdir -pv /{boot,home,mnt,opt,srv}
mkdir -pv /etc/{opt,sysconfig}
mkdir -pv /lib/firmware
mkdir -pv /media/{floppy,cdrom}
mkdir -pv /usr/{,local/}{include,src}
mkdir -pv /usr/lib/locale
mkdir -pv /usr/local/{bin,lib,sbin}
mkdir -pv /usr/{,local/}share/{color,dict,doc,info,locale,man}
mkdir -pv /usr/{,local/}share/{misc,terminfo,zoneinfo}
mkdir -pv /usr/{,local/}share/man/man{1..8}
mkdir -pv /var/{cache,local,log,mail,opt,spool}
mkdir -pv /var/lib/{color,misc,locate}

ln -sfv /run /var/run
ln -sfv /run/lock /var/lock

install -dv -m 0750 /root
install -dv -m 1777 /tmp /var/tmp
```

> **Avertissement FHS :** Le répertoire `/usr/lib64` ne doit **pas** exister dans LFS. LFS utilise uniquement `/usr/lib` pour les bibliothèques 64 bits. Si un paquet le crée, ça brisera la résolution des bibliothèques dynamiques.

9.5 Fichiers essentiels

```
ln -sv /proc/self/mounts /etc/mtab  # Table des montages, maintenue par le
noyau

cat > /etc/passwd << "EOF"
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/dev/null:/usr/bin/false
daemon:x:6:6:Daemon User:/dev/null:/usr/bin/false
messagebus:x:18:18:D-Bus Message Daemon User:/run/dbus:/usr/bin/false
nobody:x:65534:65534:Unprivileged User:/dev/null:/usr/bin/false
EOF

cat > /etc/group << "EOF"
root:x:0:
bin:x:1:daemon
sys:x:2:
tty:x:5:
daemon:x:6:
disk:x:8:
input:x:24:
mail:x:34:
wheel:x:97:
users:x:999:
nogroup:x:65534:
EOF

# Utilisateur temporaire pour les suites de tests du chapitre 8
echo "tester:x:101:101::/home/tester:/bin/bash" >> /etc/passwd
echo "tester:x:101:" >> /etc/group
install -o tester -d /home/tester

exec /usr/bin/bash --login  # Relancer bash pour résoudre "I have no name!"

touch /var/log/{btmp,lastlog,faillog,wtmp}
chgrp -v utmp /var/log/lastlog
chmod -v 664  /var/log/lastlog
chmod -v 600  /var/log/btmp
```

9.6 Outils additionnels (§7.7–7.13)

Ces 6 paquets complètent l'environnement du chroot avant la grande construction du chapitre 8. Contrairement aux paquets du chapitre 6 (compilés en cross-compilation avec `--host=$LFS_TGT`), ceux-ci sont compilés **nativement dans le chroot** — GCC et les outils temporaires sont maintenant installés et opérationnels dans `/usr`.

Paquet	Version	Utilité dans le chroot
Gettext	1.0	`msgfmt`, `msgmerge`, `xgettext` — requis par Glibc (système final)
Bison	3.8.2	Générateur de parseurs — requis par GCC, Glibc

Paquet	Version	Utilité dans le chroot
Perl	5.42.0	Requis par de nombreux systèmes de build (Autoconf, Intltool…)
Python	3.14.3	Requis par Meson, Ninja, et les paquets Python
Texinfo	7.2	Génération de documentation (pages info)
Util-linux	2.41.3	Utilitaires système (`mount`, `fdisk`, `blkid`…)

Particularité Gettext : Seuls 3 programmes sont nécessaires maintenant (`msgfmt`, `msgmerge`, `xgettext`). La version complète de Gettext sera installée au chapitre 8 (§8.34).

Nettoyage final (§7.13) — libère ~3 Go :

```
rm -rf /usr/share/{info,man,doc}/*
find /usr/{lib,libexec} -name \*.la -delete
rm -rf /tools
```

> **Attention :** La suppression de `/tools` est irréversible. C'est le répertoire de la chaîne croisée temporaire du chapitre 5 — elle n'a plus aucune utilité une fois dans le chroot et le chapitre 8 peut commencer.

---

10. Construction du système final — Chapitre 8

10.1 Vue d'ensemble

Le chapitre 8 est la phase centrale du projet LFS : construire le système de base complet en compilant **83 paquets** dans le chroot. Contrairement aux chapitres 5 et 6 qui produisaient des outils temporaires ou croisés, tout ce qui est construit ici constitue le **système LFS définitif**.

Chaque paquet est extrait, configuré, compilé, testé et installé directement dans `/usr` — sans `DESTDIR`, sans `$LFS`, directement dans la racine du chroot qui *est* le futur système. Après ce chapitre, un système Linux fonctionnel existe dans `$LFS`.

Caractéristique	Valeur
Nombre de paquets	83
Durée estimée	~80 SBU (variable selon le matériel)
Espace disque pendant le build	~6–8 Go dans `/sources`
Contexte d'exécution	Chroot uniquement
Script associé	`06_system_build.sh`

> **Règle générale sur les tests :** Les `make check` sont lancés pour tous les paquets qui les supportent. Des échecs de tests sont **normaux et acceptables** pour GCC, Bash, Perl, Autoconf, Automake — leurs suites de tests exercent des fonctionnalités du noyau non disponibles dans le

> chroot isolé (namespaces, certains appels système…). Ce qui compte, c'est que l'installation elle-même réussisse.

10.2 Tableau des 83 paquets

§	Paquet	Version	Rôle
8.3	man-pages	6.17	Pages de manuel Linux
8.4	iana-etc	20260202	`/etc/services` et `/etc/protocols`
8.5	Glibc	2.43	Bibliothèque C standard (système final)
8.6	Zlib	1.3.2	Compression zlib — dépendance de nombreux paquets
8.7	Bzip2	1.0.8	Compression bzip2
8.8	Xz	5.8.2	Compression LZMA/XZ
8.9	Lz4	1.10.0	Compression rapide LZ4 — requis par Systemd
8.10	Zstd	1.5.7	Compression Zstandard — requis par Systemd, noyau
8.11	File	5.46	Identification du type de fichiers
8.12	Readline	8.3	Édition de ligne interactive (Bash, Python…)
8.13	PCRE2	10.47	Expressions régulières compatibles Perl — requis par Grep, Systemd
8.14	M4	1.4.21	Processeur de macros
8.15	Bc	7.0.3	Calculateur de précision arbitraire
8.16	Flex	2.6.4	Générateur d'analyseurs lexicaux
8.17	Tcl	8.6.17	Langage de script Tcl (requis par Expect, DejaGNU)
8.18	Expect	5.45.4	Automatisation d'interactions terminal — requis par les tests GCC
8.19	DejaGNU	1.6.3	Framework de tests — requis par les tests GCC, Binutils
8.20	Pkgconf	2.5.1	Outil de configuration de paquets (`pkg-config`)
8.21	Binutils	2.46.0	Assembleur, linker, outils binaires (système final)
8.22	GMP	6.3.0	Bibliothèque arithmétique de précision multiple
8.23	MPFR	4.2.2	Arithmétique flottante multi-précision
8.24	MPC	1.3.1	Arithmétique complexe multi-précision
8.25	Attr	2.5.2	Attributs étendus de fichiers
8.26	Acl	2.3.2	Listes de contrôle d'accès
8.27	Libcap	2.77	Capacités POSIX (capabilities Linux)
8.28	Libxcrypt	4.5.2	Bibliothèque de hachage de mots de passe (yescrypt)
8.29	Shadow	4.19.3	Gestion des comptes utilisateurs (passwd, useradd, su…)

§	Paquet	Version	Rôle
8.30	GCC	15.2.0	Compilateur C/C++ — système final
8.31	Ncurses	6.6	Bibliothèques interfaces texte
8.32	Sed	4.9	Éditeur de flux
8.33	Psmisc	23.7	`pstree`, `killall`, `fuser`
8.34	Gettext	1.0	Internationalisation (gettext complet)
8.35	Bison	3.8.2	Générateur de parseurs (système final)
8.36	Grep	3.12	Recherche dans les fichiers
8.37	Bash	5.3	Shell (système final, avec Readline)
8.38	Libtool	2.5.4	Scripts de build pour bibliothèques partagées
8.39	GDBM	1.26	Base de données clé/valeur GNU
8.40	Gperf	3.3	Générateur de tables de hachage parfaites
8.41	Expat	2.7.4	Parseur XML
8.42	Inetutils	2.7	Outils réseau basiques (`hostname`, `ping`, `traceroute`)
8.43	Less	692	Visualiseur de fichiers (remplace `more`)
8.44	Perl	5.42.0	Langage de script Perl (système final)
8.45	XML::Parser	2.47	Module Perl pour XML — requis par Intltool
8.46	Intltool	0.51.0	Extraction de chaînes internationalisables
8.47	Autoconf	2.72	Génération de scripts `configure`
8.48	Automake	1.18.1	Génération de `Makefile.in`
8.49	OpenSSL	3.6.1	Cryptographie TLS/SSL
8.50	Libelf	0.194	Lecture/écriture de fichiers ELF (depuis elfutils)
8.51	Libffi	3.5.2	Interface d'appel de fonctions étrangères
8.52	SQLite	3.51.2	Base de données SQL embarquée
8.53	Python	3.14.3	Langage Python (système final, avec optimisations)
8.54	Flit-Core	3.12.0	Backend de build Python (pip)
8.55	Packaging	26.0	Bibliothèque Python pour les métadonnées de paquets
8.56	Wheel	0.46.3	Format de distribution Python
8.57	Setuptools	82.0.0	Outils de packaging Python
8.58	Ninja	1.13.2	Système de build rapide — requis par Meson
8.59	Meson	1.10.1	Système de build moderne — requis par Systemd, D-Bus

§	Paquet	Version	Rôle
8.60	Kmod	34.2	Gestion des modules noyau (`modprobe`, `lsmod`, `rmmod`)
8.61	Coreutils	9.10	Utilitaires de base (système final, avec patch i18n)
8.62	Diffutils	3.12	`diff`, `cmp`
8.63	Gawk	5.3.2	Traitement de texte AWK
8.64	Findutils	4.10.0	`find`, `xargs`, `locate`
8.65	Groff	1.23.0	Formatage de documents (pages man)
8.66	GRUB	2.14	Grand Unified Bootloader (binaires — config en ch.10)
8.67	Gzip	1.14	Compression gzip
8.68	IPRoute2	6.18.0	Outils réseau modernes (`ip`, `ss`, `tc`)
8.69	Kbd	2.9.0	Gestion des claviers et polices console
8.70	Libpipeline	1.5.8	Manipulation de pipelines de processus
8.71	Make	4.4.1	Outil de build
8.72	Patch	2.8	Application de patches
8.73	Tar	1.35	Archivage de fichiers
8.74	Texinfo	7.2	Système de documentation GNU (système final)
8.75	Vim	9.2.0078	Éditeur de texte
8.76	MarkupSafe	3.0.3	Bibliothèque Python — dépendance de Jinja2
8.77	Jinja2	3.1.6	Moteur de templates Python — requis par Systemd
8.78	Systemd	259.1	Init system et gestionnaire de services
8.79	D-Bus	1.16.2	Bus de communication inter-processus
8.80	Man-DB	2.13.1	Base de données des pages man (`man`, `whatis`, `apropos`)
8.81	Procps-ng	4.0.6	`ps`, `top`, `free`, `vmstat`
8.82	Util-linux	2.41.3	Utilitaires système (système final : `mount`, `lsblk`, `fdisk`…)
8.83	E2fsprogs	1.47.3	Outils ext2/ext3/ext4 (`fsck`, `mke2fs`, `tune2fs`)

10.3 Paquets notables

Glibc (§8.5) — Deuxième installation, définitive

La Glibc est compilée une deuxième fois, cette fois directement dans `/usr` sans `DESTDIR`. Cette installation est définitive : c'est la vraie bibliothèque C du système LFS. Elle configure aussi les fuseaux horaires, génère les locales, et crée `/etc/nsswitch.conf` qui indique à Glibc dans quel ordre chercher les utilisateurs, les hôtes, les services (fichiers locaux, puis systemd-resolved pour le réseau).

GCC (§8.30) — ~37 SBU, le plus long

Le compilateur définitif. Les tests sont lancés sous l'utilisateur `tester` car certains tests GCC créent des fichiers temporaires qui pourraient interférer si lancés en root. Après installation, un sanity check vérifie que le compilateur produit bien des binaires utilisant le chargeur dynamique LFS et non celui de l'hôte.

Shadow (§8.29) — Sécurité des mots de passe

Shadow configure YESCRYPT comme algorithme de hachage (plus résistant aux attaques par dictionnaire que l'ancien SHA-512). Le mot de passe root par défaut est `lfsroot`.

> **Attention de sécurité :** Ce mot de passe doit être **immédiatement changé** après le premier démarrage avec `passwd root`.

Systemd (§8.78) — Le gestionnaire de système

Systemd est à la fois le processus init (PID 1) et le gestionnaire de services. Il remplace l'ancien SysV init et prend en charge le démarrage parallèle des services, la journalisation, la gestion du réseau, et bien d'autres fonctions. LFS 13.0-systemd utilise Systemd comme composant central du système. Après installation, `systemd-machine-id-setup` génère un identifiant unique pour la machine, et `systemctl preset-all` active les services configurés comme actifs par défaut.

D-Bus (§8.79) — Communication inter-processus

D-Bus est le bus de messages qui permet aux processus de communiquer entre eux de façon standardisée. Systemd, les applications graphiques (GNOME, KDE si installés plus tard), et de nombreux services système s'appuient dessus. D-Bus doit être installé *après* Systemd car il a besoin que l'identifiant machine soit déjà généré.

10.4 Nettoyage et stripping

Après les 83 paquets, une phase de stripping supprime les symboles de debug des binaires et bibliothèques, réduisant la taille du système de 200 à 400 Mo. Les symboles des bibliothèques critiques (Glibc, libstdc++) sont préservés dans des fichiers `.dbg` séparés pour le débogage éventuel.

```
find /usr/lib -type f -name \*.so* ! -name \*dbg \
    -exec strip --strip-unneeded {} \;
find /usr/{bin,sbin,libexec} -type f \
    -exec strip --strip-all {} \;
```

---

11. Configuration du système — Chapitre 9

Le chapitre 9 donne son identité au système : son nom sur le réseau, sa langue, son fuseau horaire, son clavier. Ce sont des fichiers de configuration statiques créés une fois et lus par systemd au démarrage. Tous ces fichiers sont créés dans le chroot via `07_system_config.sh`.

Paramètres à adapter avant d'exécuter le script :

```
LFS_HOSTNAME="lfs"           # Nom d'hôte du système
LFS_TIMEZONE="Europe/Paris"  # Fuseau horaire
LFS_LANG="fr_FR.UTF-8"       # Locale
LFS_KEYMAP="fr-latin9"       # Disposition clavier console
NET_INTERFACE="eth0"         # Interface réseau (vérifier avec 'ip link')
NET_DHCP="true"              # true=DHCP, false=IP statique
```

11.1 Réseau (systemd-networkd)

LFS utilise `systemd-networkd` pour la gestion du réseau. C'est la solution intégrée à systemd, légère et suffisante pour un système de base. Elle se configure via des fichiers dans `/etc/systemd/network/` — un fichier par interface réseau.

Configuration DHCP (la plus courante pour une VM) :

```
# /etc/systemd/network/10-eth-dhcp.network
[Match]
Name=eth0

[Network]
DHCP=ipv4

[DHCPv4]
UseDomains=true
```

Configuration IP statique (si nécessaire) :

```
# /etc/systemd/network/10-eth-static.network
[Match]
Name=eth0

[Network]
Address=192.168.1.100/24
Gateway=192.168.1.1
DNS=192.168.1.1
```

`systemd-resolved` gère la résolution DNS. Le lien symbolique suivant le connecte à la configuration réseau :

```
ln -sfv /run/systemd/resolve/resolv.conf /etc/resolv.conf
```

> **Attention :** Le nom d'interface (`eth0`) peut différer selon l'environnement. Dans les VMs modernes sous systemd, les interfaces ont des noms stables comme `enp0s3` (VirtualBox) ou `ens3`

> (QEMU/KVM). Vérifier avec `ip link` depuis l'hôte avant de configurer. Pour activer au premier démarrage : `systemctl enable systemd-networkd systemd-resolved`.

11.2 Hostname et /etc/hosts

```
echo "lfs" > /etc/hostname
```

```
# /etc/hosts
127.0.0.1  localhost
127.0.1.1  lfs
::1        localhost ip6-localhost ip6-loopback
ff02::1    ip6-allnodes
ff02::2    ip6-allrouters
```

L'entrée `127.0.1.1` associe le nom de la machine à une adresse loopback. C'est nécessaire pour que certaines applications (PAM, Kerberos, certains serveurs) puissent résoudre le hostname localement sans connexion réseau active.

11.3 Horloge et fuseau horaire

```
# /etc/adjtime — Mode de l'horloge matérielle
0.0 0 0.0
0
UTC
```

Ce fichier indique à systemd que l'horloge BIOS/UEFI est réglée en UTC. C'est la configuration recommandée car elle évite les problèmes avec les changements d'heure saisonniers. Le fuseau horaire local est configuré séparément via un lien symbolique :

```
ln -sfv /usr/share/zoneinfo/Europe/Paris /etc/localtime
```

> **Attention VM :** Dans VirtualBox ou QEMU avec un hôte Windows, l'horloge matérielle peut être en heure locale. Dans ce cas, remplacer `UTC` par `LOCAL` dans `/etc/adjtime`.

11.4 Console virtuelle

```
# /etc/vconsole.conf
KEYMAP=fr-latin9
FONT=Lat2-Terminus16
```

`vconsole.conf` configure la console Linux (les terminaux TTY, pas les émulateurs de terminaux graphiques). `KEYMAP` définit la disposition du clavier — `fr-latin9` est le clavier AZERTY français avec les caractères spéciaux comme `€`. `FONT` choisit la police d'affichage — `Lat2-Terminus16` supporte les caractères accentués latins à une taille lisible.

11.5 Locale

```
# /etc/locale.conf
LANG=fr_FR.UTF-8
```

La locale détermine la langue des messages des applications, le format des dates et des nombres, et l'encodage des caractères utilisé. Elle doit correspondre à une locale générée lors de l'installation de Glibc. UTF-8 est l'encodage universel moderne, recommandé pour tout système neuf.

11.6 /etc/inputrc

`/etc/inputrc` configure readline, la bibliothèque d'édition de ligne utilisée par Bash et de nombreux autres programmes interactifs. Ce fichier définit le comportement des touches spéciales (Début, Fin, Suppr, PgUp/PgDn), active le support des caractères 8 bits (nécessaire pour les accents en UTF-8), et désactive le bip sonore. Sans ce fichier, les touches de navigation dans le shell ne fonctionneraient pas correctement.

11.7 /etc/shells

```
# /etc/shells
/bin/sh
/bin/bash
```

Ce fichier liste les shells considérés comme valides pour les comptes utilisateurs. Il est consulté par `chsh` (changement de shell) et par certains services comme SSH ou FTP qui refusent les connexions avec un shell non listé dans ce fichier.

---

12. Kernel et Bootloader — Chapitre 10

Ce chapitre rend le système démarrable de façon autonome. Jusqu'ici, le système LFS existe sur la partition `/dev/sda3` mais ne peut pas démarrer seul — il n'y a ni noyau ni bootloader. Le chapitre 10 installe le noyau Linux compilé depuis les sources, configure GRUB pour démarrer dessus, et crée le fichier `fstab` pour que le noyau sache quoi monter au démarrage.

> **Paramètres critiques à vérifier** avant d'exécuter `08_kernel_boot.sh` :
>
> ```
> ROOT_DEV="/dev/sda3"    # Notre partition LFS — à vérifier
> impérativement
> GRUB_DISK="/dev/sda"    # Disque sur lequel installer GRUB (MBR)
> GRUB_PART_NUM="3"       # Numéro de partition dans la syntaxe GRUB
> ```

12.1 /etc/fstab

`fstab` (filesystem table) est lu par le noyau au démarrage pour savoir quelles partitions monter et comment. Sans ce fichier, le noyau ne saurait pas où est le système de fichiers racine après le pivot root.

```
# /etc/fstab
# file system    mount-point  type    options    dump  fsck
/dev/sda3        /            ext4    defaults   1     1
#/dev/sdaX       swap         swap    pri=1      0     0
```

La colonne `fsck` avec la valeur `1` indique que cette partition est vérifiée en priorité au démarrage. La partition root doit toujours être `1`. Les autres partitions utilisent `2` pour être vérifiées après.

> **Attention :** Une entrée incorrecte dans `/etc/fstab` rend le système non-démarrable. En cas de doute, utiliser l'UUID plutôt que le nom de périphérique, car le nom peut changer si un disque est ajouté : `UUID=$(blkid -s UUID -o value /dev/sda3)`.

12.2 Compilation du kernel

(~8–15 SBU selon le matériel et la configuration)

Le noyau Linux est le composant central du système d'exploitation. Il gère les processus, la mémoire, les périphériques, et fait le lien entre le hardware et les logiciels. On le compile depuis les sources officielles pour contrôler exactement quelles fonctionnalités sont incluses.

```
tar -xf linux-6.18.10.tar.xz && cd linux-6.18.10
make mrproper      # Nettoyage absolu avant de commencer
make defconfig     # Configuration par défaut adaptée à l'architecture
courante
make -j$(nproc)
make modules_install

install -v -m755 arch/x86/boot/bzImage /boot/vmlinuz-6.18.10-lfs-13.0-
systemd
install -v -m644 System.map             /boot/System.map-6.18.10
install -v -m644 .config                /boot/config-6.18.10
```

`make defconfig` génère une configuration générique qui couvre la grande majorité des cas d'usage. Pour une VM, c'est généralement suffisant. Pour du hardware physique ou si des fonctionnalités spécifiques sont nécessaires, `make menuconfig` permet une configuration interactive avec une interface texte.

Fichier installé	Description
`vmlinuz-*`	Le noyau compressé — chargé en mémoire par GRUB au démarrage
`System.map-*`	Table des adresses des symboles du noyau — utile pour le débogage
`config-*`	La configuration utilisée — permet de reproduire le même build

> **Attention VM VirtualBox :** La `defconfig` devrait suffire, mais si le système ne démarre pas et reste bloqué sur "waiting for root device", vérifier dans la configuration du kernel que les pilotes `SATA/AHCI` et le pilote réseau `E1000` sont bien compilés en dur (pas en modules).

> **Attention UEFI :** Notre configuration utilise GRUB en mode BIOS/MBR. Si la machine utilise UEFI, l'installation de GRUB est différente et nécessite une partition ESP en FAT32. Ne pas utiliser `grub-install /dev/sda` sur un système UEFI.

12.3 Installation de GRUB

GRUB (Grand Unified Bootloader) est le premier programme exécuté par le BIOS au démarrage. Il est installé dans le MBR (Master Boot Record) du disque, les 512 premiers octets, et a la responsabilité de charger le noyau Linux en mémoire et de lui passer le contrôle.

```
grub-install /dev/sda   # Installe GRUB dans le MBR de /dev/sda
```

Le fichier de configuration `/boot/grub/grub.cfg` indique à GRUB quoi démarrer :

```
set default=0
set timeout=5

insmod part_msdos   # Table de partitions MBR (pas GPT)
insmod ext2         # Système de fichiers ext4 (ext2 supporte aussi ext4 en
lecture)
set root=(hd0,3)    # Premier disque, troisième partition

menuentry "GNU/Linux, Linux 6.18.10-lfs-13.0-systemd" {
    linux /boot/vmlinuz-6.18.10-lfs-13.0-systemd root=/dev/sda3 ro
}
```

Le paramètre `root=/dev/sda3 ro` passé au noyau est crucial : il indique au noyau Linux quelle partition monter comme racine (`/`). `ro` signifie "read-only" — le noyau monte la racine en lecture seule initialement pour permettre à `fsck` de la vérifier, puis systemd la remonte en lecture/écriture.

> **Attention sur la numérotation :** GRUB numérote les disques depuis 0 (`hd0` = premier disque) et les partitions depuis 1. Donc `(hd0,3)` = `/dev/sda3`. Ne pas confondre avec la numérotation Linux qui commence aussi à 1 pour les partitions.

12.4 Fichiers d'identification

Ces trois fichiers permettent aux logiciels et scripts d'identifier la distribution :

```
echo "13.0-systemd" > /etc/lfs-release   # Format LFS natif
```

`/etc/lsb-release` et `/etc/os-release` suivent des standards adoptés par les distributions modernes. `os-release` est lu par systemd, les gestionnaires de paquets, et des outils comme `neofetch` pour afficher les informations de la distribution.

---

13. Démontage et redémarrage — Chapitre 11

Avant de redémarrer, il faut quitter le chroot et démonter proprement tous les systèmes de fichiers virtuels depuis l'hôte. Si on redémarre brutalement sans démonter, le noyau peut forcer un `fsck` au prochain démarrage, et dans le pire des cas, des écritures en suspens peuvent ne pas être vidées sur le disque.

```
exit   # Quitter le chroot

# Depuis l'hôte, en tant que root
umount -v $LFS/dev/pts
mountpoint -q $LFS/dev/shm && umount -v $LFS/dev/shm
umount -v $LFS/dev
umount -v $LFS/run
umount -v $LFS/proc
umount -v $LFS/sys
umount -v $LFS       # Démontage de la partition LFS elle-même

reboot
```

L'ordre de démontage est important : les systèmes imbriqués (`/dev/pts` dans `/dev`) doivent être démontés avant leur parent. Tenter de démonter `/dev` alors que `/dev/pts` est encore monté échouera avec `target is busy`.

> **Si un démontage échoue :** Identifier le processus qui utilise encore le système de fichiers avec `lsof +D $LFS` ou `fuser -mv $LFS`. Terminer ce processus avant de réessayer.

Après le redémarrage, si tout s'est bien passé :

GRUB affiche le menu de démarrage, le noyau se charge, systemd s'initialise (les services démarrent en parallèle), et un prompt de login apparaît sur le TTY1 :

```
Linux From Scratch 13.0-systemd (tty1)
lfs login: root
Password: lfsroot
```

Premières actions après le premier démarrage :

```
passwd root                              # Changer le mot de passe root
immédiatement
systemctl enable systemd-networkd        # Activer le réseau au démarrage
systemctl enable systemd-resolved        # Activer la résolution DNS au
```

```
démarrage
systemctl start  systemd-networkd         # Démarrer le réseau maintenant
```

---

14. État des scripts et ordre d'exécution

Tous les scripts sont rédigés et prêts. Le build réel s'est arrêté après Glibc (§5.5) — la prochaine étape est `03_temp_tools.sh`.

Script	Couverture LFS	Contexte d'exécution
`00_download.sh`	Téléchargement de ~90 sources + 5 patches	root ou lfs
`01_host_setup.sh`	Ch.2–4 : hôte, partition, user lfs	root
`02_toolchain.sh`	Ch.5 : Binutils/GCC pass1, Linux headers, Glibc, Libstdc++	lfs
`03_temp_tools.sh`	Ch.6 : 17 paquets (M4 → GCC pass2)	lfs
`04_chroot_prep.sh`	Ch.7.2–7.6 : montages VFS, FHS, passwd/group	root
`05_chroot_tools.sh`	Ch.7.7–7.13 : Gettext, Bison, Perl, Python, Texinfo, Util-linux	**chroot**
`06_system_build.sh`	Ch.8 complet : 83 paquets + stripping	**chroot**
`07_system_config.sh`	Ch.9 : réseau, locale, horloge, clavier, shell	**chroot**
`08_kernel_boot.sh`	Ch.10 : kernel 6.18.10, GRUB, fstab, release files	**chroot**
`09_unmount.sh`	Ch.11 : démontage VFS avant reboot	root (hôte)

Ordre d'exécution complet

```
# ── Sur l'hôte, en tant que root ──────────────────────────────
export LFS=/mnt/lfs
mount -v -t ext4 /dev/sda3 $LFS

bash scripts/00_download.sh          # Télécharger les sources
bash scripts/01_host_setup.sh        # Si pas encore fait

# ── En tant qu'utilisateur lfs ────────────────────────────────
su - lfs
bash scripts/02_toolchain.sh         # ~17 SBU — Binutils/GCC pass1, Glibc,
Libstdc++
bash scripts/03_temp_tools.sh        # ~12 SBU — Outils temporaires ch.6
exit

# ── Sur l'hôte, en tant que root ──────────────────────────────
bash scripts/04_chroot_prep.sh setup   # Changer proprio + structure FHS +
copier scripts
```

```
bash scripts/04_chroot_prep.sh mount   # Monter les VFS
bash scripts/04_chroot_prep.sh enter   # Entrer dans le chroot

# ── Dans le chroot ──────────────────────────────────────────────
bash /sources/scripts/05_chroot_tools.sh   # ~3 SBU  — §7.7-7.13
bash /sources/scripts/06_system_build.sh   # ~80 SBU — Ch.8 (83 paquets)
bash /sources/scripts/07_system_config.sh  # < 1 SBU — Ch.9
bash /sources/scripts/08_kernel_boot.sh    # ~12 SBU — Ch.10 (kernel +
GRUB)
exit

# ── Sur l'hôte, en tant que root ────────────────────────────────
bash scripts/09_unmount.sh
reboot
```

Avant de reprendre le build

```
export LFS=/mnt/lfs
mount -v -t ext4 /dev/sda3 $LFS
cp -r scripts/ $LFS/sources/   # Rendre les scripts accessibles dans le
chroot
su - lfs
bash scripts/03_temp_tools.sh
```

> **Rappel des pièges critiques :**
>
> 1. Toujours `echo $LFS` avant toute opération — doit retourner `/mnt/lfs`.
> 2. `su - lfs` avec le tiret pour sourcer `.bash_profile`.
> 3. Scripts 05→08 uniquement depuis le chroot — jamais depuis l'hôte.
> 4. Remonter les VFS (`04_chroot_prep.sh mount`) à chaque reprise après redémarrage.
> 5. Changer le mot de passe root après le premier démarrage.

Document rédigé à partir des notes de construction et du livre LFS 13.0-systemd (avril 2026)